98. Передача по ссылке

Хотя **передача по значению** является хорошим вариантом во многих случаях, она имеет несколько ограничений. Во-первых, при передаче по значению большой **структуры** или класса в функцию, создается копия аргумента и уже эта копия передается в параметр функции. В большинстве своем это бесполезная трата ресурсов, которая снижает производительность.

Во-вторых, при передаче аргументов по значению, единственный способ вернуть значение обратно в вызывающий объект — это использовать возвращаемое значение функции. Но иногда случаются ситуации, когда нужно, чтобы функция изменила значение переданного аргумента. Передача по ссылке решает все эти проблемы.

Оглавление:

1. **Передача по ссылке**
2. **Возврат сразу нескольких значений**
3. **Передача по константной ссылке**
4. **Плюсы и минусы передачи по ссылке**

Передача по ссылке

При **передаче переменной по ссылке** нужно просто объявить параметры функции как **ссылки**, а не как обычные переменные:

```
void func(int &x) // x - это переменная-ссылка
{
    x = x + 1;
}
```

При вызове функции переменная `x` станет ссылкой на аргумент. Поскольку ссылка на переменную обрабатывается точно так же, как и сама переменная, то любые изменения, внесенные в ссылку, приведут к изменениям исходного значения аргумента! В следующем примере это хорошо проиллюстрировано:

```
#include <iostream>

void boo(int &value)
{
    value = 7;
}

int main()
{
    int value = 6;

    std::cout << "value = " << value << '\n';
    boo(value);
    std::cout << "value = " << value << '\n';
    return 0;
}
```

Эта программа точно такая же, как и программа из предыдущего урока, за исключением того, что параметром функции boo() теперь является ссылка вместо обычной переменной.

Результат выполнения программы:

```
value = 6
value = 7
```

Как вы можете видеть, функция изменила значение аргумента `c` `6` на `7`!

Вот еще один пример:

```
#include <iostream>

void addOne(int &x) // x - это переменная-ссылка
{
    x = x + 1;
} // x уничтожается здесь

int main()
{
    int a = 7;
    std::cout << "a = " << a << '\n';
    addOne(a);
    std::cout << "a = " << a << '\n';
    return 0;
}
```

Результат выполнения программы:

```
a = 7
a = 8
```

Обратите внимание, значение аргумента `a` было изменено функцией.

Возврат сразу нескольких значений

Иногда нам может понадобиться, чтобы функция возвращала сразу несколько значений. Однако оператор return позволяет функции иметь только одно возвращаемое значение. Одним из способов возврата сразу нескольких значений является использование ссылок в качестве параметров:

```
#include <iostream>
#include <math.h> // для sin() и cos()

void getSinCos(double degrees, double &sinOut, double &cosOut)
{
    // sin() и cos() принимают радианы, а не градусы, поэтому необходима конвертация
    const double pi = 3.14159265358979323846; // значение Пи
    double radians = degrees * pi / 180.0;
    sinOut = sin(radians);
    cosOut = cos(radians);
}

int main()
{
    double sin(0.0);
    double cos(0.0);

    // функция getSinCos() возвратит sin и cos в переменные sin и cos
    getSinCos(30.0, sin, cos);

    std::cout << "The sin is " << sin << '\n';
    std::cout << "The cos is " << cos << '\n';
    return 0;
}
```

Эта функция принимает один параметр (передача по значению) в качестве входных данных и «возвращает» два параметра (передача по ссылке) в качестве выходных данных. Параметры, которые используются только для возврата значений обратно в caller, называются **параметрами вывода**. Они дают понять caller-у, что значения исходных переменных, переданных в функцию, не столь значительны, так как мы ожидаем, что эти переменные будут перезаписаны.

Давайте рассмотрим это детально. Во-первых, в функции main() мы создаем локальные переменные `sin` и `cos`. Они передаются в функцию getSinCos() по ссылке (а не по значению). Это означает, что функция getSinCos() имеет прямой доступ к исходным значениям переменных `sin` и `cos`, а не к их копиям. Функция getSinCos(), соответственно, присваивает новые значения переменным `sin` и `cos` (через ссылки `sinOut` и `cosOut`), перезаписывая их старые значения. Затем main() выводит эти обновленные значения.

Если бы `sin` и `cos` были переданы по значению, а не по ссылке, то функция getSinCos() изменила бы копии `sin` и `cos`, а не исходные значения и эти изменения уничтожились бы в конце функции — переменные вышли бы из **локальной области видимости**. Но, поскольку `sin` и `cos` передавались по ссылке, любые изменения, внесенные в `sin` или `cos` (через ссылки), сохраняются и за пределами функции getSinCos(). Таким образом, мы можем использовать этот механизм для возврата сразу нескольких значений обратно в caller.

Хотя этот способ хорош, но он также имеет свои нюансы. Во-первых, синтаксис немного непривычен, так как параметры ввода и вывода указываются вместе с вызовом функции. Во-вторых, в caller-е не очевидно, что `sin` и `cos` являются параметрами вывода, и они будут изменены функцией. Это, вероятно, самая опасная часть данного способа передачи (так как может привести к ошибкам). Некоторые программисты считают, что это достаточно большая проблема, и не советуют передавать аргументы по ссылке, отдав предпочтение передаче по адресу, не смешивая при этом параметры ввода и вывода.

Лично я не рекомендую смешивать параметры ввода и вывода именно по этой причине, но если вы это делаете, то обязательно добавляйте **комментарии к коду**, описывая, что вы делаете и как это делаете.

Неконстантные ссылки могут ссылаться только на неконстантные **l-values** (например, на неконстантные переменные), поэтому параметр-ссылка не может принять аргумент, который является константным l-value или r-value (например, литералом или результатом выражения).

Передача по константной ссылке

Одним из самых главных недостатков передачи по значению является то, что все аргументы, переданные по значению, *копируются* в параметры функции. Когда аргументами являются большие структуры или классы, то этот процесс может занять много времени. В случае с передачей по ссылке эта проблема легко решается. Когда аргумент передается по ссылке, то создается ссылка на фактический аргумент (что занимает минимальное количество времени на выполнение), и никакого копирования значений не происходит. Это позволяет передавать большие структуры или классы с минимальной затратой ресурсов.

Однако здесь также могут возникнуть потенциальные проблемы. Ссылки позволяют функции изменять значения аргументов напрямую, что нежелательно, если мы хотим, чтобы аргумент был доступен только для чтения. Когда мы знаем, что функция не должна изменять значение аргумента, но не хотим использовать передачу по значению, то лучшим решением будет использовать **передачу по константной ссылке**.

Вы уже знаете, что **константная ссылка** — это ссылка на переменную, значение которой изменить через эту же ссылку не получится никак. Следовательно, если мы используем константную ссылку в качестве параметра, то получаем 100% гарантию того, что функция не изменит аргумент!

Запустив следующий фрагмент кода, мы получим ошибку компиляции:

```
void boo(const int &y) // y - это константная ссылка
{
    y = 8; // ошибка компиляции: константная ссылка не может изменить свое же значение!
}
```

Использование const полезно по нескольким причинам:

→ Мы получаем гарантию от компилятора, что значения, которые не должны быть изменены — не изменятся (компилятор выдаст ошибку, если мы попытаемся сделать нечто подобное тому, что было в вышеприведенном примере).

→ Программист, видя const, понимает, что функция не изменит значение аргумента. Это может помочь при **отладке программы**.

→ Мы не можем передать константный аргумент в неконстантную ссылку-параметр. Использование константного параметра гарантирует, что мы сможем передавать как неконстантные, так и константные аргументы в функцию.

→ Константные ссылки могут принимать любые типы аргументов, включая l-values, константные l-values и r-values.

Правило: При передаче аргументов по ссылке всегда используйте константные ссылки, если вам не нужно, чтобы функция изменяла значения аргументов.

Плюсы и минусы передачи по ссылке

Плюсы передачи по ссылке:

+ Ссылки позволяют функции изменять значение аргумента, что иногда полезно. В противном случае, для гарантии того, что функция не изменит значение аргумента, нужно использовать константные ссылки.

+ Поскольку при передаче по ссылке копирования аргументов не происходит, то этот способ гораздо эффективнее и быстрее передачи по значению, особенно при работе с большими структурами или классами.

+ Ссылки могут использоваться для возврата сразу нескольких значений из функции (через параметры вывода).

Минусы передачи по ссылке:

− Трудно определить, является ли параметр, переданный по неконстантной ссылке, параметром ввода, вывода или того и другого одновременно. Разумное использование const и суффикса `Out` для внешних переменных решает эту проблему.

− По вызову функции невозможно определить, будет аргумент изменен функцией или нет. Аргумент, переданный по значению или по ссылке, выглядит одинаково. Мы можем определить способ передачи аргумента только просмотрев объявление функции. Это может привести к ситуации, когда программист не сразу поймет, что функция изменяет значение аргумента.

Когда использовать передачу по ссылке:

→ при передаче структур или классов (используйте const, если нужно только для чтения);

→ когда нужно, чтобы функция изменяла значение аргумента.

Когда не использовать передачу по ссылке:

→ при передаче фундаментальных типов данных (используйте передачу по значению);

→ при передаче обычных **массивов** (используйте передачу по адресу).